

Week 2: Docker 기초와 실행 환경 표준화

Overview

2주차는 Week 1에서 만든 로컬 실행 evidence를 Docker 이미지와 컨테이너로 옮기는 주간이다. 학생은 "내 컴퓨터에서는 실행된다"를 넘어서, 같은 실행 조건을 다른 사람도 재현할 수 있도록 Dockerfile, image, container, port binding, volume, environment variable, Compose로 정리한다.

이번 주의 중심 질문은 다음과 같다.

애플리케이션 실행 조건을 이미지와 컨테이너로 표준화하면 실행, 확인, 중지, 복구, 인수인계가 어떻게 달라지는가?

Docker는 운영 문제를 모두 없애는 도구가 아니다. 실행 환경을 더 명시적으로 포장하고, 실행 조건을 문서와 명령으로 재현 가능하게 만드는 도구다. 따라서 이번 주의 평가는 명령어 암기가 아니라 실행 증거, 장애 기록, README handoff, 비용/보안/정리 기준을 기준으로 한다.

처음이면 여기부터

Week 2는 Docker가 반드시 필요하다. macOS는 Docker Desktop을 기본 경로로 진행하고, Linux는 Docker Desktop 또는 Docker Engine 중 하나를 선택한다. 설치가 끝난 뒤에는 "앱을 열었다"가 아니라 CLI evidence로 확인한다.

- macOS/Linux 설치 절차: [필수 소프트웨어 설치 가이드](#)
- Docker 최소 확인 명령: `docker version`, `docker compose version`, `docker run --rm hello-world`
- Linux에서 권한 문제가 나면 `sudo docker run --rm hello-world` 성공 여부와 Docker post-install 적용 여부를 구분해 기록한다.
- Docker Hub password, access token, MFA code는 README, terminal history, screenshot에 남기지 않는다.

Learning Goals

- Docker가 해결하려는 실행 환경 차이, 배포 재현성, 의존성 충돌 문제를 설명한다.
- image와 container의 차이를 파일 묶음과 실행 중인 process 관점으로 구분한다.
- `docker run`, `docker ps`, `docker logs`, `docker stop`, `docker rm` 으로 컨테이너 lifecycle을 확인한다.
- Dockerfile을 작성해 표준 실습 애플리케이션 이미지를 직접 빌드한다.
- port binding, environment variable, volume, network를 이용해 컨테이너 실행 조건을 제어한다.
- Docker Compose로 웹 애플리케이션과 데이터베이스를 함께 실행하고 상태를 확인한다.
- 로그, HTTP status, container status, README 기록으로 장애 원인을 분석한다.

Weekly Keywords

- Docker
- image
- container
- Docker Desktop
- Docker Engine
- registry
- Docker Hub
- Dockerfile
- layer
- tag
- digest
- port binding
- volume
- bind mount
- named volume
- environment variable
- bridge network
- Docker Compose
- service

- `compose.yaml`
- container logs

Schedule Index

- Day 1: 1주차 복습, Docker Desktop 설치, Docker 개념, 기본 명령어, hello-world/nginx/표준 앱 첫 실행, 환경 점검
- Day 2: 이미지와 레이어, Dockerfile 문법, 표준 실습 앱 빌드, build 문제 해결, 태그와 Docker Hub
- Day 3: 컨테이너 네트워크, 포트 바인딩, 환경변수, 볼륨, DB 컨테이너, 로그 기반 장애 분석
- Day 4: Docker Compose 필요성, `compose.yaml` , 웹 앱 + DB 실행, Compose 네트워크, 개발 환경 구성, 보충 실습
- Day 5: Docker 운영 관점, 좋은 Dockerfile, 보안 기초, 이미지 배포 흐름, 통합 실습, 학습 정리 제출, 다음 주차 연결

Week 1 To Week 2 Mapping

Week 1 local evidence	Week 2 Docker 표현	운영 판단
app folder	build context, COPY 범위	이미지에 필요한 파일만 포함했는가
start command	CMD, docker run command	컨테이너가 어떤 process로 시작되는가
localhost port	-p host:container port binding	외부 사용자가 어느 port로 접근하는가
data path	bind mount, named volume	컨테이너 삭제 후 데이터가 남아야 하는가
config note	-e, .env, Compose environment	설정을 이미지에 굳히지 않았는가
log evidence	docker logs, docker compose logs	장애 증거를 어디서 확인하는가
README run step	Docker build/run/compose section	다른 사람이 실행 절차를 재현할 수 있는가
RCA note	Docker failure analysis	port conflict, env missing, volume reset을 기록했는가

Required Deliverables

- 표준 실습 애플리케이션용 `Dockerfile`
- `compose.yaml` 1개
- Docker build/run/compose 실행 명령이 포함된 `README.md`
- Docker Hub 또는 표준 registry에서 내려받아 실행한 이미지 1개
- 직접 빌드한 로컬 이미지 tag 1개
- port, log, environment variable, volume 중 하나 이상을 포함한 장애 분석 기록 1개
- 3주차 학습 전 Docker readiness checklist

Assessments

- [Week 2 객관식 문제 세트](#)

Practice Environment

항목	기준
OS	기본 실습은 macOS 기준. Windows 사용자는 WSL 2/가상화/권한 예외 경로를 별도 기록
Docker Desktop	공식 문서 기준 설치, 실행, 로그인 또는 pull 가능 상태 확인
CLI	docker version, docker run hello-world, docker ps 실행 가능
Browser/curl	nginx 또는 표준 앱 HTTP 응답 확인
Repository	Dockerfile, compose, README, RCA note를 저장할 GitHub repository
비용	Week 2는 cloud resource를 만들지 않는다. 로컬 Docker와 Docker Hub pull/push 중심으로 진행한다.

Official References

Topic	Reference	확인할 키워드
Docker overview	https://docs.docker.com/engine/docker-overview/	image, container, registry, Docker Engine
Docker Desktop	https://docs.docker.com/desktop/	Desktop, GUI, build/share/run
Mac install	https://docs.docker.com/desktop/setup/install/mac-install/	Apple silicon, Intel, system requirements
Windows install	https://docs.docker.com/desktop/setup/install/windows-install/	Windows 사용자의 WSL 2, system requirements, permissions
Image concept	https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/	immutable, layers, package
Dockerfile	https://docs.docker.com/guides/docker-concepts/building-images/writing-a-dockerfile/	FROM, WORKDIR, COPY, RUN, CMD
Docker Compose	https://docs.docker.com/compose/	multi-container, services, networks, volumes
Compose services	https://docs.docker.com/reference/compose-file/services/	ports, environment, volumes, networks
Compose networking	https://docs.docker.com/compose/how-tos/networking/	service name, DNS, network

Cost And Security Notes

- Week 2 실습은 로컬 Docker 중심이므로 AWS, Kubernetes cluster, 유료 cloud database를 만들지 않는다.
- Docker Desktop 라이선스 조건은 조직 규모와 사용 목적에 따라 달라질 수 있으므로 공식 문서와 교육 운영 기준을 확인한다.
- Docker Hub password, access token, MFA code는 README, screenshot, terminal history에 남기지 않는다.
- DB password는 실습용이어도 공개 repository에 그대로 올리지 않는다. 공개해야 할 것은 값이 아니라 "어떤 환경변수 이름이 필요한가"와 "어디에 기록하지 않는가"다.
- 실습 후 불필요한 container, image, volume을 정리해 host disk 사용량을 관리한다.

Weekly Checklist

- ☐ `docker version` 결과를 기록했다.
- ☐ `docker run hello-world` 성공 또는 실패 증상을 기록했다.
- ☐ `docker run -p 8080:80 nginx` 실행 후 browser 또는 `curl` 로 확인했다.
- ☐ 표준 실습 앱 이미지를 pull 또는 build했다.
- ☐ Dockerfile 의 각 instruction이 하는 일을 설명했다.
- ☐ `compose.yaml` 로 웹 앱과 DB를 실행했다.
- ☐ logs/status/HTTP response 중 하나 이상으로 정상 상태를 확인했다.
- ☐ port conflict, env missing, volume issue 중 하나를 RCA 형식으로 기록했다.

Glossary

2주차 용어는 [glossary.md](#)를 기준으로 정리한다. 용어 정의는 명령어 암기보다 "어떤 상태를 확인하거나 어떤 운영 위험을 줄이는가"에 연결해 읽는다.

Next Week Connection

3주차의 MSA는 여러 서비스가 API와 network로 연결되는 구조를 다룬다. Week 2에서 Compose로 웹 앱과 DB를 함께 실행해 본 경험은 서비스 분리, service name, network boundary, configuration handoff를 이해하는 선행 경험이 된다.